LECTURE 3

Bottom Up Explicit Search

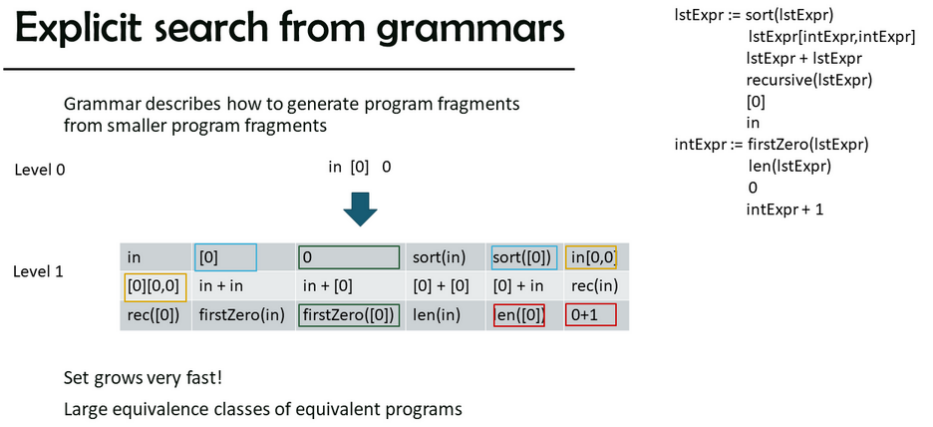
3.1 Simple bottom up search

The simplest bottom up synthesis algorithm works by explicitly constructing all possible programs from a grammar starting with the terminals in the language. As one can imagine, this can be very inefficient, since the space of all expressions grows very large even with very small programs. The key idea behind this algorithm is to prune the set of primitives at every step by eliminating those that are deemed to be "*observationally equivalent*"; i.e. those which produce the same outputs on those inputs that were given as a specification. The key ideas of this algorithm were first presented in a paper by Albarghouthi, Gulwani and Kinkaid [1], although a very similar algorithm was discovered independently and presented only a few months later by Udupa et al. [2].

The high-level algorithm is shown below.

```
Synthesize(inputs, outputs):
  plist := set of all terminals
  while(true):
    plist := grow(plist);
    plist := elimEquvalents(plist, inputs);
    forall( p in plist)
      if(isCorrect(p, inputs, outputs)): return p;
```

The key steps in the algorithm are the `grow` operation, which uses the non-terminals in the grammar to construct new terms from all the terms in `plist`, and the `elimEquivalents` step, which eliminates all terms that are deemed to be redundant by virtue of being equivalent to other terms in the list. A key idea behind this algorithm is that the check of equivalence is not an real equivalence check, which would be expensive. Instead, the expressions are tested on the target inputs, and any two expression that produce the same outputs on these inputs are deemed equivalent, regardless of whether they are truly equivalent or not. This is what is referred to as "observational equivalence", the idea being that since we only care about the behavior of the synthesized program on the given inputs, any behavior difference on other inputs is irrelevant.



As an example, consider again the language introduced earlier. Initially, the `plist` will contain all the terminals in the grammar: `in`, `[0]`, `0`. After the first call to `grow`, the set of expressions grows quite dramatically, as it now includes all expressions that can be creating by composing the original terminals using the different production rules in the grammar. As we can see in the figure, however, many of these expressions are clearly equivalent. For example, `sort([0])` is equivalent to `[0]`, and `firstZero([0])` is equivalent to `0`. More interestingly, `in[0,0]` and `[0][0,0]` may not be equivalent in general, but if I only have two inputs `[0,7,3,2,5,6,3]` and `[0,2,13,5,9,1,0]`, then `in[0,0]` and `[0][0,0]` are observationally equivalent, since they are equivalent on all available inputs; on any program that uses `in[0,0]`, we can replace it with `[0][0,0]`, and it will produce the same output on the given inputs.

For each equivalence class of observationally equivalent programs, `elimEquivalents` eliminates all but one of them. Each equivalent program that is eliminated actually leads to exponential savings, since it spares us from having to construct the exponentially many programs that could be constructed from that sub-program.

This algorithm as described is extremely simple, but it is already quite powerful. First, it naturally explores small programs before large programs, so it automatically finds the smallest program satisfying the specification. Additionally, it is easy to introduce heuristics into `grow` and `elimEquivalents` to direct the search for programs so that programs that are deemed more desirable are discovered first, or to speed up the search based on prior knowledge of which programs are more likely to be correct. A second benefit of this algorithm is that it works with black-box language building blocks. For example, for the language above, there is no need to have source code for `sort` or `firstZero`; the algorithm just needs to be able to execute them. The algorithm also does a good job of coping with symmetries, and is able to exploit properties of the building blocks even without access to their source code; for example, even

without being told that `sort` is idempotent, the algorithm would immediately discard `sort(sort(in))` after deeming it equivalent to `sort(in)`. However, the algorithm is not a magic bullet, there are some important conditions that need to be met for the algorithm to work correctly, conditions that our language in the running example doesn't actually meet!

3.2 Formal Requirements

The bottom-up search algorithm comes with some important formal requirements. The most important requirement is that the semantics of a program fragment should not depend on the context. The algorithm relies on being able to prune an expression e_2 on the basis of there being another expression e_1 with the same semantics already in the search space. But this only works if the following condition is satisfied:

Context independent equivalence: Given two expressions e_1 and e_2 evaluated on inputs σ, bottom up search requires that the following condition be true: $\forall\sigma.ObsEquiv(e_1, e_2, \sigma) \Rightarrow \forall C.ObsEquiv(C[e_1], C[e_2], \sigma)$

Where $ObsEquiv(e_1, e_2, inputs)$ is the observational equivalence of two expressions under a given set of inputs.

In the notation $C[e]$, C is a *context*, i.e. an expression with a hole, and $C[e]$ is the expression you get by filling the hole in C with the expression e. In other words, if two expressions e_1 and e_2 are equivalent for a given set of inputs, then any expression that includes e_1 as a sub-expression will be equivalent to the same expression with e_2 replacing e_1.

As an example, the property clearly holds for arithmetic expressions. For example, the expression $(x + 2)/5$ equals $(x - 2)$ when $x = 3$. Therefore, in any expression that includes $(x + 2)/5$ as a sub-expression we can replace $(x + 2)/5$ by $(x - 2)$ and its value will remain unchanged for $x = 3$. This follows from the fact that arithmetic expressions are evaluated in a bottom up fashion irrespective of their context. On the other hand, the property will generally not hold for imperative programs. For example, even though the expressions above are equal when $x = 3$, the programs below will not be equal even when fed the value $x = 3$.

```
def f1(x):
    x = x + 5
    return (x+2)/5

def f2(x):
    x = x + 5
    return (x-2)
```

With this in mind, the language from the running example should be raising alarm bells. This is because the language semantics are not context inde-

pendent. In particular, the value produced by the recursive call depends on the context of the call (since the call will return the result of evaluating the entire program on that input). For example, consider the expressions `rec(in[1, len(in)])` and `[]`. For the input list `[1,2,3]`, the two expressions evaluate to the empty list. But they will not be equivalent in all contexts. In particular, the expressions `rec(in[1, len(in)])+[0]` and `[]+[0]` will produce very different outcomes. Does this mean that we will not be able to use bottom-up search with observational equivalence on this language?

Not quite, but we will have to be more careful in defining the notion of observational equivalence that will be used to prune the search space; doing this correctly can be quite tricky. For example, one seemingly quick fix is to simply enforce that expressions with `rec` will never be observationally equivalent to anything else. This will prevent the context sensitive calls from being merged without affecting the ability to merge context insensitive calls. This will handle the case where e_1 and e_2 have `rec` calls, but will **not** guarantee context independent equivalence. Why not? Because the context may also have calls to `rec`. So for example, consider the expressions `in[0,0]` and `[0]`. For the input `[0,1,2,3]` the two expressions are the same, but under that input, the expressions `rec(in[1, len(in)])+in[0,0]` and `rec(in[1, len(in)])+[0]` are not equivalent.

A different way to salvage bottom up search for this language, or for any language that involves recursive calls, is to require *trace completeness*. This means that you require that the user provided examples include examples for any recursive call. For example, if I want to provide examples for the `reverse` function, I would provide the example `[1,2,3]=>[3,2,1]` but trace completeness would also require me to provide the example `[2,3]=>[3,2]`. Now, when constructing examples bottom-up, any expression that includes `rec` can be checked to see if there is an example with those arguments. If there is, then we know from the example what the result of the `rec` call should be, so the call does not actually need to be evaluated. And if the arguments are not in the list of examples, the expression can be ruled out.

Trace completeness may sound like a cop-out, and in some ways it is. For programming by example problems, requiring trace completeness may be asking too much from the user. But there are many applications of program synthesis, for example for reverse engineering, where it is relatively cheap to request additional examples, so for those applications, trace completeness may be an adequate solution.

Even when context indepedent equivalence is satisfied, there is another important limitation of the algorithm: scalability. Even with very aggressive prunning it is hard to scale the algorithm beyond a handful of terms. Moreover, while the algorithm does a good job at discovering how to connect discrete components, it does very poorly when there is a need to discover constants in

a program, whether integers or reals.

3.3 Bottom-up search with probabilities

Recall from the last lecture that we can frame program synthesis as a posterior inference problem, in which we seek the program $p*$ that maximizes the posterior probability $P(p \mid e)$, where e is some evidence (specification) provided by the user. In particular, suppose that e is a set of input-output examples and we take the likelihood function to be $p(e \mid p) = 1$ if p produces the correct output for all examples e, and 0 otherwise. As mentioned in the previous lecture, this allows us to subsume the PBE problem, since any program that does not match the examples exactly will have zero posterior probability. However, finding the maximum-a-posteriori (MAP) program $p*$ is rarely tractable with such a sharp likelihood function, which sparked a wave of research in the late 2010s on using deep learning to fit approximate posteriors $p_\theta(p \mid e)$. However, it turns out that bottom-up search can also be used to find the MAP program in a few interesting cases of the prior $P(p)$, two of which we will discuss now.

3.3.1 Length minimization

Without making any assumptions about the problem, one common choice of prior in machine learning (and statistics) is to follow the minimum description length (MDL) principle. This principle essentially states that the prior probability of a hypothesis should be inversely proportional to its size. In our context, a priori we should prefer smaller programs over larger ones, since they are less likely to be overfitting the examples.

As it turns out, the bottom-up search algorithm described earlier is actually a search for the MAP program under the MDL prior! That is, the program which bottom-up yields is provably the one that maximizes the posterior probability $P(p \mid e)$, if the likelihood is defined as above and the prior is inversely proportional to the depth of the program. The only requirement is that when two observationally equivalent programs are found, the algorithm must always keep the shortest one.

It is important to note, however, that some of the improvements to bottom-up search that will be described later in the lecture mean the resulting algorithm no longer guarantees length minimization. For example, the STUN approach needs to trade off the size of the solution in a given branch with the strength of the predicate guarding that branch, and that tradeoff may lead to programs that are not minimal.

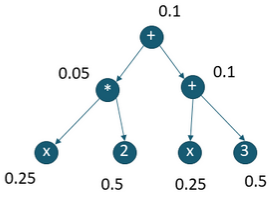
3.3.2 Search with probabilistic grammars

Probabilistic Grammars

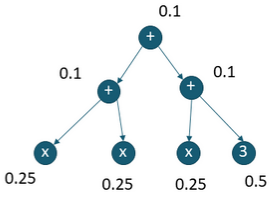
	P(expr)
expr = expr + expr	0.1
| expr – expr	0.1
| expr * expr	0.05
| *var*	0.25
| *num*	0.5

Probabilistic Grammars

	P(expr)
expr = expr + expr	0.1
| expr – expr	0.1
| expr * expr	0.05
| *var*	0.25
| *num*	0.5



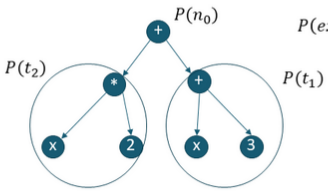
P(exp) = 7.81×10^{-6}



P(exp) = 7.81×10^{-6}

Search with P G

Probability is compositional



$P(expr) = P(n_0) * P(t_1) * P(t_2)$

Same strategies as length-based metric work

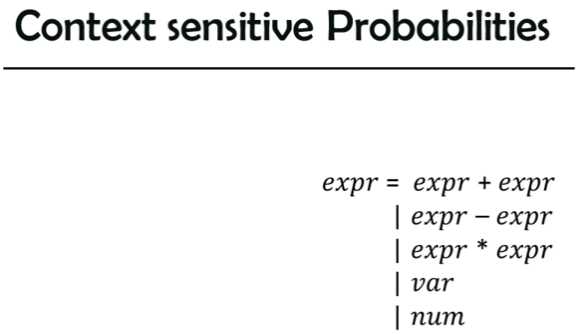
The approaches for length minimization we have described so far work with relatively few changes for some richer representations of the distribution. For example, one kind of representation involves probabilistic grammars. A simple

form of a probabilistic grammar is one where every production has a probability, and the probabilities of all the productions for a given non-terminal add up to one. Under this formalism, the probability of an expression generated by the grammar is the product of the probabilities of all the production rules used to create the expression.

Generating an expression that maximizes the probability under this scheme is very similar to generating an expression that minimizes length. From the perspective of bottom-up search, distributions of this kind have the advantage that they are compositional and context insensitive. This means that for every equivalence class of behaviorally equivalent expressions we only need keep the one with the maximum probability. How do we know that by keeping only the expression with maximum probability we are not missing out on an expression that may prove useful later?

As a proof we know that for any expression exp' we throw away, there is an equivalent expression exp that we kept that has the same input/output behavior as exp' and the same or higher probability. So we know that the exp' cannot be necessary to construct the highest probability expression, because any expression $C[exp']$ that uses exp' as a sub-expression can be replaced by $C[exp]$ to yield the same input/output behavior with the same or higher probability.

3.4 Context sensitive probabilistic grammars



Context sensitive Probabilities

$$
\begin{aligned}
expr = \ & expr_1 + expr_2 && \text{A}\\
| \ & expr_3 - expr_4 && \text{B}\\
| \ & expr_5 * expr_6 && \text{C}\\
| \ & var && \text{D}\\
| \ & num && \text{E}
\end{aligned}
$$

	A	B	C	D	E
Root	0.2	0.2	0.2	0.2	0.2
1	0.2	0.2	0.2	0.2	0.2
2	0	0	1/3	1/3	1/3
3	0.2	0.2	0.2	0.2	0.2
4	0	0	1/3	1/3	1/3
5	0.2	0.2	0.2	0.2	0.2
6	0.25	0.25	0	0.25	0.25

Bottom-up search

x=5 => out=7



We know they are equivalent
Which one do we keep?

Bottom-up search

$$
\begin{aligned}
expr = \ & expr_1 + expr_2 && \text{A}\\
| \ & expr_3 - expr_4 && \text{B}\\
| \ & expr_5 * expr_6 && \text{C}\\
| \ & var && \text{D}\\
| \ & num && \text{E}
\end{aligned}
$$

	A	B	C	D	E
Root	0.2	0.2	0.2	0.2	0.2
1	0.2	0.2	0.2	0.2	0.2
2	0	0	1/3	1/3	1/3
3	0.2	0.2	0.2	0.2	0.2
4	0	0	1/3	1/3	1/3
5	0.2	0.2	0.2	0.2	0.2
6	0.25	0.25	0	0.25	0.25

x=5 => out=7



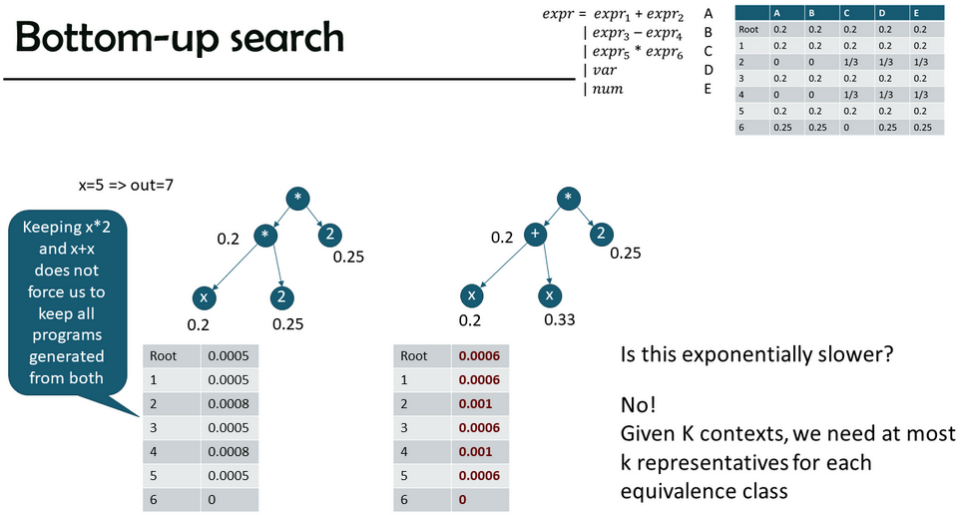
Root	0.01
1	0.01
2	**0.017**
3	0.01
4	**0.017**
5	0.01
6	0



Root	**0.013**
1	**0.013**
2	0
3	**0.013**
4	0
5	**0.013**
6	**0.0165**

Is this exponentially slower?

No!
Given K contexts, we need at most k representatives for each equivalence class



	A	B	C	D	E
Root	0.2	0.2	0.2	0.2	0.2
1	0.2	0.2	0.2	0.2	0.2
2	0	0	1/3	1/3	1/3
3	0.2	0.2	0.2	0.2	0.2
4	0	0	1/3	1/3	1/3
5	0.2	0.2	0.2	0.2	0.2
6	0.25	0.25	0	0.25	0.25

A richer kind of probabilistic grammar is one where the probabilities depend on the context as illustrated in the figure. In this setting, the probability of a given production depends on where the production is used. This kind of probabilistic grammar poses a problem for bottom-up search, because given two equivalent ASTs, it may no longer be possible to determine that one has strictly higher probability than the other. For example, consider the case illustrated in the figure. The probability of the expressions $x * 2$ and $x + x$ depends on the context. In some contexts, the first one will have a higher probability and in other contexts, the second one will, so if the bottom-up algorithm wants to be able to produce the highest probability expression it needs to keep them both.

Now, one of the advantages of eliminating expressions based on observational equivalence is that eliminating a single expression early on can have exponential benefits, since it also eliminates the exponentially large number of expressions that can be constructed from that expression. So an important question is: Now that we are forced to keep multiple equivalent expressions, will that make the algorithm exponentially more expensive? The answer is surprisingly NO!

To understand why, consider again the example with the two expressions, $x * 2$ and $x + x$. For every expression that is constructed with $x * 2$, there will now be an equivalent expression that uses $x + x$ instead. However, we will not actually be keeping all these equivalent expressions. An expression only needs to be kept if there is some context in which it is better than all other equivalent expressions. This means that for every equivalence class, we need to keep at most N expressions, where N is the number of distinct contexts in the grammar. This is worse than the original algorithm which only had to keep one representative from every equivalence class, but only by a constant factor.

3.5 Synthesis through Unification (STUN)

One way to address the scalability challenge is to modularize the search. One way to do this is that rather than trying to synthesize a program that works for all inputs in one shot, one can search for multiple programs that work for different situations and then find a way of combining them together into a program that works for all inputs. This idea was first formalized by Alur, Cerny and Radhakrishna [3] in an algorithm called *Synthesis through Unification*(STUN).

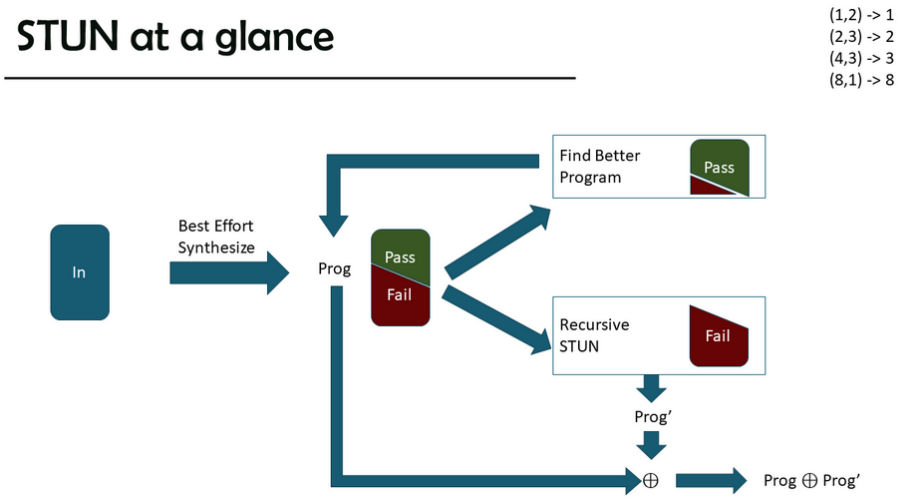
To illustrate the idea, suppose you are given the following input output pairs:

```
input   ⟶ output
(4, 5)  ⟶ 4
(12,14) ⟶ 12
(13, 10)⟶ 10
(5, 1)  ⟶ 1
```

And suppose you are given the following grammar:

$intExpr$	:=	fst	first element of the input pair
		snd	second element of the input pair
		0	constant zero
		$intExpr + 1$	adds one to a number
		$if(boolExpr)\ intExpr\ else\ intExpr$	conditional expression
$boolExpr$	:=	$intExpr > intExpr$	integer comparison
		$boolExpr \,\&\, boolExpr$	conjunction

The simple bottom-up search would require enumerating a fairly large set of programs, before hitting on the correct program, but we can also observe that the program fst actually works for half of the given inputs, and the program `snd` works for all the others. Moreover, we can see that all the inputs on which `fst` produces the correct answer have the property that `fst < snd`, so from that we can construct the program `if(fst < snd) fst else snd`. The STUN approach makes this intuition systematic, by providing a strategy for synthesizing programs that work for subsets of the inputs and then discovering how to combine them into a complete program that works for all inputs. The general strategy works even for cases where we cannot simply introduce arbitrary branches, but there is also a more specialized version of the approach that works particularly well in cases where you can introduce branches [4].



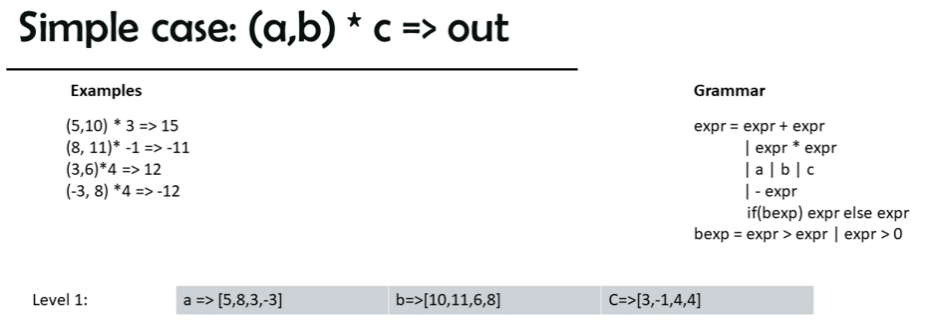
The figure on the right illustrates a simplified version of the full algorithm. The procedure takes as input a set of inputs on which we wish the program to work correctly. As a first step, we do a first best-effort synthesis attempt and produce a program that works correctly on some inputs and incorrectly on others (if we happen to get a program that works on all inputs, we are done and we can just return that program). If the produced program fails on some input, we have two choices, we can either try to improve on the current program by demanding that it continues to work correctly on the current set of good inputs, but also on some of the other inputs on which it currently fails, or we can recursively call the `STUN` procedure on those inputs on which the current program `Prog` does not work. This will produce a new program `Prog'`, which works on all those inputs. Then the two programs `Prog` and `Prog'` need to be unified into a single program that works for all inputs.

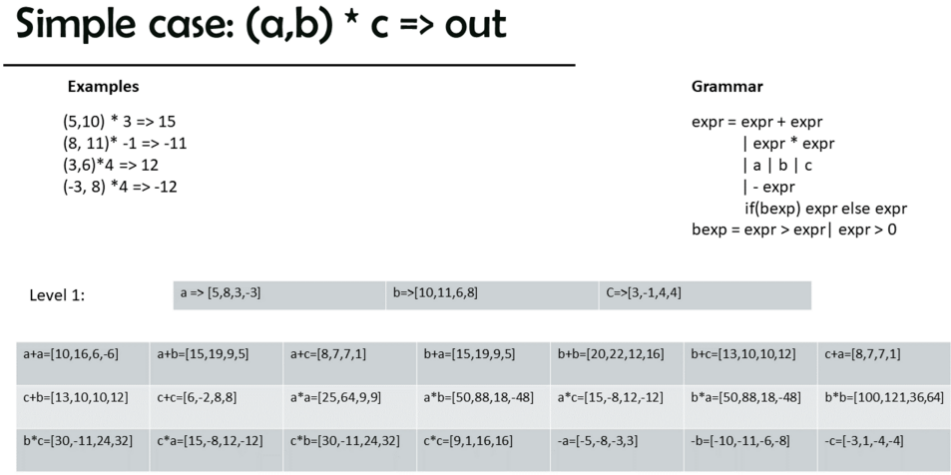
The full algorithm also needs to deal with the scenario where either the recursive call to STUN or the generation of a better program fail to find a program. In some cases, unification may require the programs to satisfy some additional conditions in order to succeed, so the algorithm needs to track those as well. Both of these are low-level details not captured by the figure. In the paper, the crucial decision of whether to try to continue to refine the current solution, or to recursively call the STUN procedure is handled by a simple heuristic: pick a random input, if that input fails, use that input to search for a better solution, if it succeeds, then perform the recursive call. This is a crude heuristic based on the intuition that the recursive call only happens when the current solution already works for a high-enough fraction of the current inputs.

This is the general framework, but any specific instance of the algorithm needs to address the question of how to actually perform the unification of the two programs that work on different sets of inputs.

3.6 Example: Arithmetic with top-level branches

For this example, consider the following problem. We have a range of values, represented by its two extremes (a, b), and we want to discover the lower bound of the new range when we multiply this range by another value c. Below are a few examples which will become the input to our algorithm.





Simple case: (a,b) * c => out

Examples

(5,10) * 3 => 15
(8, 11)* -1 => -11
(3,6)*4 => 12
(-3, 8) *4 => -12

Eliminate Observationally equivalent ones

Grammar

expr = expr + expr
| expr * expr
| a | b | c
| - expr
|if(bexp) expr else expr
bexp = expr > expr | expr > 0

Level 1:

a => [5,8,3,-3]	b=>[10,11,6,8]	C=>[3,-1,4,4]

a+a=[10,16,6,-6]	a+b=[15,19,9,5]	a+c=[8,7,7,1]	b+a=[15,19,9,5]	b+b=[20,22,12,16]	b+c=[13,10,10,12]	c+a=[8,7,7,1]
c+b=[13,10,10,12]	c+c=[6,-2,8,8]	a*a=[25,64,9,9]	a*b=[50,88,18,-48]	a*c=[15,-8,12,-12]	b*a=[50,88,18,-48]	b*b=[100,121,36,64]
b*c=[30,-11,24,32]	c*a=[15,-8,12,-12]	c*b=[30,-11,24,32]	c*c=[9,1,16,16]	-a=[-5,-8,-3,3]	-b=[-10,-11,-6,-8]	-c=[-3,1,-4,-4]

Simple case: (a,b) * c => out

Examples

(5,10) * 3 => 15
(8, 11)* -1 => -11
(3,6)*4 => 12
(-3, 8) *4 => -12

Identify an expression that works for a subset of the inputs

Grammar

expr = expr + expr
| expr * expr
| a | b | c
| - expr
| if(bexp) expr else expr
bexp = expr > expr | expr > 0

Level 1:

a => [5,8,3,-3]	b=>[10,11,6,8]	C=>[3,-1,4,4]

a+a=[10,16,6,-6]	a+b=[15,19,9,5]	a+c=[8,7,7,1]	b+a=[15,19,9,5]	b+b=[20,22,12,16]	b+c=[13,10,10,12]	c+a=[8,7,7,1]
c+b=[13,10,10,12]	c+c=[6,-2,8,8]	a*a=[25,64,9,9]	a*b=[50,88,18,-48]	a*c=[15,-8,12,-12]	b*a=[50,88,18,-48]	b*b=[100,121,36,64]
b*c=[30,-11,24,32]	c*a=[15,-8,12,-12]	c*b=[30,-11,24,32]	c*c=[9,1,16,16]	-a=[-5,-8,-3,3]	-b=[-10,-11,-6,-8]	-c=[-3,1,-4,-4]

Simple case: (a,b) * c => out

Examples

(5,10) * 3 => 15
(8, 11)* -1 => -11
(3,6)*4 => 12
(-3, 8) *4 => -12

Identify an expression that works for the rest of the inputs

Grammar

expr = expr + expr
| expr * expr
| a | b | c
| - expr
| if(bexp) expr else expr
bexp = expr > expr| expr > 0

Level 1:

a => [5,8,3,-3]	b=>[10,11,6,8]	C=>[3,-1,4,4]

a+a=[10,16,6,-6]	a+b=[15,19,9,5]	a+c=[8,7,7,1]	b+a=[15,19,9,5]	b+b=[20,22,12,16]	b+c=[13,10,10,12]	c+a=[8,7,7,1]
c+b=[13,10,10,12]	c+c=[6,-2,8,8]	a*a=[25,64,9,9]	a*b=[50,88,18,-48]	a*c=[15,-8,12,-12]	b*a=[50,88,18,-48]	b*b=[100,121,36,64]
b*c=[30,-11,24,32]	c*a=[15,-8,12,-12]	c*b=[30,-11,24,32]	c*c=[9,1,16,16]	-a=[-5,-8,-3,3]	-b=[-10,-11,-6,-8]	-c=[-3,1,-4,-4]

Simple case: (a,b) * c => out

Examples

(5,10) * 3 => 15
(3,6)*4 => 12
(-3, 8) *4 => -12
a*c P_1

(8, 11)* -1 => -11
b*c P_2

Grammar

expr = expr + expr
| expr * expr
| a | b | c
| - expr
| if(bexp) expr else expr
bexp = expr > expr| expr > 0

a>a=[f,f,f,f]	a>b=[f,f,f,f]	a>c=[t,f,f,t]
b>a=[t,t,t,t]	b>b=[f,f,f,f]	b>c=[t,t,t,t]
c>a=[f,t,t,f]	c>b=[f,f,f,f]	c>c=[f,f,f,f]
a>0=[t,t,f,t]	b>0=[t,t,t,t]	c>0=[t,t,t,f]

$$P_1 \oplus P_2 = if(c > 0)\ a * c\ else\ \ b * c$$

```
(5,10) * 3 => 15
(8, 11)* -1 => -11
(3,6)*4 => 12
(-3, 8) *4 => -12
```

The goal is to synthesize a small program that computes this given the following grammar.

$$\begin{aligned} expr &= expr + expr \\ &\quad | \, expr * expr \\ &\quad | \, a \mid b \mid c \mid -expr \\ &\quad if(bexp)\ expr\ else\ expr \\ bexp &= expr > expr \mid expr > 0 \end{aligned}$$

In the animation above, it is possible to see how after reaching only depth 2 of an explicit search, we can already find an expression that works for a subset of the inputs. In fact, we can find expressions that together work for all the inputs. So now, the challenge is to unify them into a single expression that works for all inputs. In this case, we do the unification by discovering a branch that separates the inputs that work correctly for one branch from the inputs that work correctly for another.

In this case, we can easily find that the expression `c > 0` precisely separates the two cases, so we can unify the two solutions into a general program `if(c > 0) a * c else b * c`.

3.7 What if the language does not have top level branches?

Consider the following alternative variant of the language above.

$$
\begin{array}{rcl}
var & = & a \mid b \mid c \\
expr & = & expr + expr \\
 & & \mid expr * expr \\
 & & \mid var \mid -expr \\
 & & if(bexp)\ var\ else\ var \\
bexp & = & expr > expr \mid expr > 0
\end{array}
$$

The language is very similar, but now we are left without the option of performing unification by introducing top-level branches. This means we need a different mechanism for performing unification, that is, for joining the two programs that work for different subsets of the input into a new program that works for all inputs. In order to do this, we will use something that in the literature is known as **antiunification** (even though the STUN paper [3] does not use that terminology).

3.7.1 Unification and Antiunification

In the literature, the term unification generically means finding a common structure for two different expressions. More precisely, however, the literature distinguishes between *Unification*, where you find the common structure by replacing variables with expressions, and *Antiunification*, where you find the common structure by replacing expressions with variables.

So for example, if I have two expressions $x + 5$ and $7 + y$, unification finds the common structure as $Unify(x + 5, 7 + y) = [7 + 5, (x- > 7, y- > 5)]$. That is, it tells us that the two expressions can be turned into the same expression by replacing x with 7 and y with 5. By contrast, if you have two expressions $7*3+2$ and $5 * 3 + 2$, antiunification identifies the common structure by introducing variables as $Antiunify(7 * 3 + 2, 5 * 3 + 2) = x * 3 + 2$.

The STUN paper does not actually use this terminology, but it is actually fairly standard, particularly in the inductive logic programming literature. So just keep in mind when reading the STUN paper that it uses Unification in the more generic sense that can potentially mean antiunification depending on the context.

Antiunification

Given two expressions $expr_1$, $expr_2$ find an expression $expr^*$ with some free variables v_i such that

- $expr_1 = expr^*[v_i \rightarrow e_i^1]$
- $expr_2 = expr^*[v_i \rightarrow e_i^2]$

$Antiunify(expr_1, expr_2) = expr^*$

In general, you want to maximally expose sharing between $expr_1$ and $expr_2$.

- You want to avoid just making $expr^* = v$

What if there is no top level branch?

Examples

(5,10) * 3 => 15 (3,6)*4 => 12 (-3, 8) *4 => -12	a*c	P_1
(8, 11)* -1 => -11	b*c	P_2

Grammar

```
var = a | b | c
expr = expr + expr
     | expr * expr
     | var
     | - expr
     | if(bexp) var else var
bexp = expr > expr| expr > 0
```

Antiunify(a*c, b*c) = v*c

Find an expression v such that

(5,10) * 3 => v=5 (3,6)*4 => v=3 (-3, 8) *4 => v=-3	(8, 11)* -1 => v=11	v=if(c>0) a else b

3.7.2 STUN without top-level branches

Without the ability to introduce branches, we need an alternative way of combining together two expressions. In the example above, once we discover that there are two expressions `a*c` and `b*c` that together cover all the inputs, we can use antiunification to produce a common expression v`*c` where v stands for a fragment of code, so now we need to discover what this missing code fragment is by recursively solving a (hopefully smaller) synthesis problem. In this case, we will discover that v=`if(c>0) a else b`.

One thing to note is that when the expression $b * c$ was discovered, the synthesizer could have just as easily discovered the expression $-b$. This would have been a problem, because whereas antiunification of $b * c$ and $a * c$ worked exactly as expected, the expression $-b$ cannot be antiunified with $a * c$ except by making the whole expression a variable. The way STUN deals with this is that when a recursive call to STUN is performed, the algorithm can also pass additional constraints that the expression discovered by the recursive call must satisfy. Therefore, when recursively calling STUN with the example `(8, 11)* -1 => -11`, we can impose the additional constraint that the discovered expression must be antiunified with $a * c$. This would force the recursive call to STUN to produce $b * c$ instead of $-b$.

The original STUN paper [3] describes other examples of the Unification operation $\oplus$. In particular, it describes an approach for bit-vectors that is also based on antiunification, but we will not describe it here.

3.8 Hierarchical Search

The approaches described earlier provide a way of modularizing the search by first discovering components that work on some fragments of the input space and then discovering how to weave them together to cover the entire space. This input-based modularization is only one of many different approaches of breaking up the search into independent search spaces. An alternative approach to improve the scalability of bottom-up search is to search the space hierarchically. This can be done in cases where the program can be split into different levels of abstraction, and where one can perform the search at each level independently.

One example of this is the recent work by Wang, Cheung and Bodik [5]. In this case, their goal is to synthesize complex SQL queries from examples. The key insight in their paper is that the problem can be decomposed in a hierarchical way. To understand how this can work, consider the following query language, which is a much simplified version of the language their system actually supports.

$$
\begin{array}{rcl}
Rel & = & T \\
 & | & \text{Select } Fields \text{ from } Rel \text{ where } Pred \\
 & | & Rel, Rel \\
Pred & = & exp = exp \\
 & | & exp > exp \\
 & | & Pred \ \& \ Pred \\
Fields & = & table.name \text{ as } name \\
 & | & table.name \text{ as } name, Fields
\end{array}
$$

In the language above, a relation is either the name of a table T, a query on

a relation, or a cross product on two different relations. Now, suppose you are given some tables, for example:

```
Employee                  Depts
---------                 -------------
Name, Dept                Dept, Building
---------                 -------------
Todd, Sales               Sales, A1
Joe, Engineering          Engineering, A2
Alice, Engineering        Operations, A1
Sally, Operations
```

And suppose you are told the expected output of a query is this:

```
Output
---
XX
---
Todd
Sally
```

The interesting thing to notice is that there are two aspects of the query that need to be discovered, and that are mostly independent of each other. On the one hand, there is the structure of the query, which is going to determine the "shape" of the output; the fact that there is a single column containing records that originally came from the Employee table. And on the other, there are the selection predicates. These include both the fact that we are selecting records where `Building=A1`, or the fact that we are joining on matching the `Dept` code.

The key idea in their hierarchical search approach is to perform the search in two levels. First, use a simple bottom-up search to discover the structure of the query, in a language that replaces all predicates with holes as shown below. For each query in that language that produces output in the right shape, we can now search to see if there is a set of predicates that can be inserted into the holes that will cause the query to produce the right set of records.

$$
\begin{array}{rcl}
Rel & = & T \\
 & | & \text{Select } Fields \text{ from } Rel \text{ where } \square \\
 & | & Rel, Rel \\
Fields & = & table.name \text{ as } name \\
 & | & table.name \text{ as } name, Fields
\end{array}
$$

An important ingredient in making this work is that we need to be able to evaluate the queries with holes and tell whether they can potentially produce an output with the right shape. The key idea is to define a semantics for queries with holes that is guaranteed to produce a superset of the records that any instantiation of the holes may produce. This is easy to do in the language above by simply treating every hole as $True$, although in the paper it gets a little more

tricky because they also support aggregates. In general, though, if the queries with holes are guaranteed to produce a superset of the results, then we can search the space of queries with holes and safely rule out any query that does not produce a superset of our desired results.

In the case of the example above, we may find that both the queries below produce results of the right shape.

```
Select name as XX from Employee where _
Select name as XX from Employee, Depts where _
```

Having found some candidate queries with holes, the synthesizer can then search for predicates that lead the queries to select the correct set of records.

Just as in the case of STUN, breaking the search into smaller search problems has a significant performance benefit. In this case, we are exploiting something about the structure of the search space, the ability to tell that a query structure will produce incorrect outputs before we even know what predicates are, in order to break the search into somewhat independent search problems. Just like in the STUN case, the partition is not perfect, so we may need to try many potential solutions to the high-level problem to see which one can actually be extended to a complete solution.

3.9 Additional reading

There is at this point an extensive literature on bottom-up search for program synthesis. An especially significant line of work relies on reframing bottom-up search in terms of Tree automata. The seminal paper in that literature is by Wang, Dillig and Singh [6], but it has been extended to a wide range of applications.

References

[1] Aws Albarghouthi, Sumit Gulwani, and Zachary Kincaid. Recursive program synthesis. In *Computer Aided Verification - 25th International Conference, CAV 2013, Saint Petersburg, Russia, July 13-19, 2013. Proceedings*, pages 934–950, 2013. doi: 10.1007/978-3-642-39799-8_67. URL https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/cav13.pdf.

[2] Abhishek Udupa, Arun Raghavan, Jyotirmoy V. Deshmukh, Sela Mador-Haim, Milo M.K. Martin, and Rajeev Alur. Transit: Specifying protocols with concolic snippets. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '13, pages 287–296, New York, NY, USA, 2013. ACM. ISBN 978-1-4503-2014-6. doi: 10.1145/2491956.2462174. URL http://doi.acm.org/10.1145/2491956.2462174.

[3] Rajeev Alur, Pavol Černý, and Arjun Radhakrishna. Synthesis through unification. In Daniel Kroening and Corina S. Păsăreanu, editors, *Computer Aided Verification*, pages 163–179, Cham, 2015. Springer International Publishing. ISBN 978-3-319-21668-3. URL http://ecee.colorado.edu/pavol/publications/cav15a/cav15a.pdf.

[4] Rajeev Alur, Arjun Radhakrishna, and Abhishek Udupa. Scaling enumerative program synthesis via divide and conquer. In Axel Legay and Tiziana Margaria, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 319–336, Berlin, Heidelberg, 2017. Springer Berlin Heidelberg. ISBN 978-3-662-54577-5. URL https://link.springer.com/chapter/10.1007/978-3-662-54577-5_18.

[5] Chenglong Wang, Alvin Cheung, and Rastislav Bodik. Synthesizing highly expressive sql queries from input-output examples. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2017, pages 452–466, New York, NY, USA, 2017. ACM. ISBN 978-1-4503-4988-8. doi: 10.1145/3062341.3062365. URL http://doi.acm.org/10.1145/3062341.3062365.

[6] Xinyu Wang, Işıl Dillig, and Rishabh Singh. Synthesis of data completion scripts using finite tree automata. *Proceedings of the ACM on Programming Languages*, 1(OOPSLA):62:1–62:26, 2017. doi: 10.1145/3133886.